Техническое задание на создание системы metroLog

Дисциплина: Разработка клиент-серверных приложений **Проект:** metroLog — система учёта средств измерений и эталонов **Студент:** Макеев Глеб Борисович **Группа:** BHEBDPS-24-RKSP-5 **Дата:** 18.10.2026 **Версия документа:** 1.0 **Репозиторий:** https://github.com/mflkee/metroLog

---

11.1 Введение

Наименование системы: metroLog — информационная система учёта средств измерений (СИ) и эталонов средств измерений (ЭСИ).

Условное обозначение: metroLog.

Основание для разработки: потребность метрологической службы в едином реестре измерительного оборудования и контроле сроков поверки. Работа выполняется в рамках дисциплины «Разработка клиент-серверных приложений»; предметная область и требования зафиксированы в отчёте по практике 1.

Заказчик: метрологическая служба (внутренний заказчик, эксплуатация «как есть»).

Исполнитель: Макеев Глеб Борисович, группа BHEBDPS-24-RKSP-5.

Сроки: практики 1–4 семестра, демонстрация — по графику дисциплины.

Источники разработки:

- регламенты поверки и ремонта средств измерений;
- требования ФГИС «Аршин» к составу сведений о поверках;
- действующая практика учёта метрологической службы (Excel-реестры, переписка);
- нормативные документы: ГОСТ 34.602-89 (техническое задание), ГОСТ 34.601-90

(стадии создания автоматизированных систем).

Термины и сокращения:

Сокращение	Расшифровка
СИ	средство измерений
ЭСИ	эталон средства измерений
ИО	испытательное оборудование
ВО	вспомогательное оборудование
ФГИС «Аршин»	Федеральная государственная информационная система «Аршин» (сведения о поверках)
RBAC	ролевое управление доступом (Role-Based Access Control)
REST	архитектурный стиль взаимодействия поверх HTTP
SPA	одностраничное веб-приложение (Single Page Application)
API	программный интерфейс приложения

11.2 Назначение и цели создания системы

Назначение. Автоматизация учёта, поверки и ремонта средств измерений и эталонов в метрологических службах и лабораториях: единый реестр оборудования, контроль сроков, прозрачная история процессов, разграничение доступа между исполнителями и заказчиками.

Цели создания:

1. Централизовать учёт СИ/ЭСИ вместо разрозненных Excel-файлов по папкам и подразделениям.
2. Исключить просроченные поверки за счёт хранения дат поверки и срока действия свидетельства.
3. Автоматизировать получение сведений из ФГИС «Аршин»: поиск, импорт, пакетное обновление папки.
4. Обеспечить единую историю поверок и ремонтов с этапами, сроками и ответственными.
5. Разграничить доступ: заказчик видит только закреплённые за ним папки.
6. Снизить ручной труд при подготовке отчётов (дашборд, выгрузка XLSX).

Критерии достижения целей:

Цель	Измеримый критерий
Единый реестр	все приборы компании в одной БД, иерархия «папка → группа → прибор»
Контроль сроков	у каждой карточки есть дата поверки и срок действия; просрочка видна в реестре
Интеграция с Аршином	поиск по свидетельству/рег. номеру и пакетное обновление папки работают без ручного ввода
История процессов	по каждому прибору доступны все поверки и ремонты с перепиской и вложениями
Разграничение доступа	пользователь без доступа к папке получает 404, а не данные
Отчётность	выгрузка реестра, очередей и журнала событий в XLSX

11.3 Характеристика объектов автоматизации

Объект автоматизации — процессы учёта и обслуживания измерительного оборудования метрологической службы:

- ведение реестра оборудования по объектам (папкам) и группам;
- постановка СИ на поверку, контроль этапов и сроков, получение свидетельств;
- заявка на ремонт, контроль маршрута и этапов, закрытие с результатом;
- сверка карточек с ФГИС «Аршин»;
- коммуникация исполнителей и заказчиков (комментарии, вложения, упоминания);
- подготовка отчётов и аудит действий.

Участники процесса:

Роль	Код	Функции
Разработчик	`DEVELOPER`	техническая поддержка и диагностика, полный доступ
Администратор	`ADMINISTRATOR`	пользователи, роли, права по папкам, справочники, настройки
Сотрудник метрологической службы	`MKAIR`	ведение реестра, поверки и ремонты, переписка, отчёты по своей зоне
Заказчик	`CUSTOMER`	просмотр закреплённых папок, согласование, комментарии к своим приборам

Масштаб: от нескольких единиц до нескольких десятков пользователей; единая база на собственном сервере; несколько тысяч карточек оборудования.

Особенности предметной области, влияющие на систему:

- данные о поверках первичны во внешнем государственном реестре — система обязана

работать и при его недоступности (кэш последних сведений);

- заводской номер не уникален: в Аршине один короткий номер может принадлежать десяткам

приборов, поэтому автоматическое сопоставление требует участия оператора;

- сроки ремонта и поверки нормативно заданы и не должны «переписываться» задним числом;
- заказчик не должен видеть приборы других организаций (soft-denial через 404).

11.4 Требования к системе

11.4.1 Требования к системе в целом

Архитектура. Трёхуровневая клиент-серверная система:

1. **Клиент** — SPA на React 19 + TypeScript, собирается Vite, раздаётся nginx.
2. **Сервер приложений** — FastAPI (Python 3.12), REST API под префиксом `/api/v1`,

бизнес-логика в сервисном слое, доступ к данным через SQLAlchemy 2.0.

1. **Хранилище** — PostgreSQL 16; Redis 7 + RQ для фоновых задач; файловое хранилище

вложений на диске сервера.

Режим работы: круглосуточный, веб-доступ через браузер; обновление — контейнерами Docker Compose с автоматическим применением миграций при старте.

Требования к интеграции: HTTP-интеграция с ФГИС «Аршин» (поиск и детализация по свидетельству и регистрационному номеру), SMTP для email-уведомлений.

11.4.2 Требования к функциям (подсистемам)

Подсистема	Состав функций
Аутентификация и профиль	вход по email/паролю, выдача токена, профиль, смена пароля, проверка SMTP
Пользователи и роли	CRUD пользователей, назначение роли, блокировка, временный пароль, скоуп папок
Реестр оборудования	папки, группы, карточки, типы СИ/ЭСИ/ИО/ВО/прочее, поиск, фильтры, сортировка, пагинация
Поверки	создание, режим проведения, этапы, сроки, переписка, групповые операции, закрытие
Ремонты	создание, маршрут, этапы, сроки, переписка, закрытие с результатом
Интеграция с Аршином	поиск, детализация, импорт, пакетное обновление папки, выбор записи при неоднозначном номере
Коммуникации	комментарии, вложения, приватные заметки, @упоминания, email-уведомления
Задачи	kanban-доска, участники и роли, чек-листы, сроки, напоминания, привязка к приборам
Отчётность и аудит	журнал событий, дашборд, экспорт XLSX, архивы вложений (ZIP)
Служебные функции	`/health`, `/health/ready`, OpenAPI-спецификация

Детализация — в приложении А (F1–F17).

11.4.3 Требования к надёжности

- Сохранность данных: PostgreSQL с транзакциями, внешние ключи с политиками `CASCADE` / `SET NULL`, резервное копирование перед каждым развёртыванием.
- Версионирование схемы: 66 миграций Alembic, применение при старте контейнера.
- Отказоустойчивость фоновых задач: при недоступности Redis уведомление отправляется синхронно (fallback), задача не теряется молча.
- Health/ready-проверки: `/health` — живость процесса, `/health/ready` — доступность БД и Redis (при `NOTIFICATION_QUEUE_ENABLED=false` Redis не требуется).
- Восстановление после сбоя: контейнеры с `restart: unless-stopped`, идемпотентные миграции и повторные запуски пакетных операций.

11.4.4 Требования к безопасности

- Обязательная аутентификация для всех прикладных эндпоинтов, кроме `/health*` и `/auth/login`.
- Пароли — только в виде хеша PBKDF2-HMAC-SHA256 (600 000 итераций, соль 16 байт).
- Токен доступа — stateless HMAC-SHA256, подписанный секретным ключом; сервер не хранит сессии.
- Ролевая модель: `DEVELOPER`, `ADMINISTRATOR`, `MKAIR`, `CUSTOMER`.

- Дополнительное ограничение по папкам: белый список разрешённых и чёрный список скрытых папок.
- **Soft-denial:** при отсутствии доступа к объекту возвращается `404`, чтобы не раскрывать

факт его существования.

- Проверка активности пользователя на каждом запросе (блокировка действует немедленно).
- Передача данных по HTTPS (TLS terminates на периметре), CORS ограничен списком origin.
- Загрузка файлов: ограничение размера (25 МБ), сжатие изображений, хранение вне web-root,

выдача только через авторизованный эндпоинт.

11.4.5 Требования к эргономике и технической эстетике

- Веб-интерфейс на русском языке, адаптивная вёрстка, тёмная и светлая темы.
- Единые паттерны: таблицы с пагинацией, диалоги подтверждения, индикаторы загрузки.
- Ошибки формулируются для пользователя, а не кодом HTTP.

11.4.6 Требования к эксплуатации, техническому обслуживанию и ремонту

- Развёртывание: Docker Compose (postgres, redis, backend, backend-worker, frontend).
- Обновление: сборка образов в CI, публикация в GHCR, перезапуск на сервере.
- Резервное копирование БД — `pg_dump` перед деплоем, хранение в каталоге резервных копий.
- Мониторинг — внешняя проверка `/api/v1/health/ready`.

11.4.7 Требования к персоналу

- Администратор: навыки управления пользователями и правами, понимание структуры папок.
- Пользователь: базовые навыки работы в браузере. Специальной подготовки не требуется.
- Разработчик: Python/FastAPI/SQLAlchemy, React/TypeScript, Docker.

11.4.8 Требования к защите информации от несанкционированного доступа

- Разграничение доступа на уровне API, а не только интерфейса.
- Приватные комментарии и сообщения видны только автору и администраторам.
- Журнал событий фиксирует действия пользователей по оборудованию, поверкам, ремонтам, задачам.
- Секреты (пароль БД, SMTP, секрет подписи токенов) — в переменных окружения, не в репозитории.

11.5 Состав и содержание работ

Этап	Содержание	Результат	Статус
1	Анализ предметной области, выявление требований, ТЗ	отчёт по практике 1, настоящее ТЗ	выполнен
2	Моделирование данных: ER-диаграмма, схема БД	ER-модель, DDL PostgreSQL	выполнен
3	Проектирование клиент-серверной архитектуры и REST API	презентация архитектуры, OpenAPI	выполнен
4	Реализация серверной части	FastAPI, SQLAlchemy, Alembic, RQ	выполнен
5	Реализация веб-клиента	React SPA, 18 экранов	выполнен
6	Тестирование	196 pytest + 33 vitest, нагрузочные тесты Locust	выполнен
7	Развёртывание и документирование	Docker Compose, CI/CD, README	выполнен
8	Демонстрация и защита	Swagger, slumber, живая система	в работе

Стадии по ГОСТ 34.601-90: техническое задание → эскизный проект (архитектура, ER) → технический проект (API, схема БД) → рабочая документация (код, тесты) → ввод в действие.

11.6 Порядок контроля и приёмки

Виды контроля:

1. **Автоматические тесты** — `pytest` (196 тестов) на API и сервисы, `vitest` (33 файла)

на клиентскую логику; запускаются в CI на каждый push.

1. **Статический анализ** — `ruff` (backend), `eslint` + `tsc --noEmit` (frontend).
2. **Нагрузочные испытания** — Locust, read-профиль, 30 и 60 одновременных пользователей.
3. **Приёмочное тестирование** — ручной сценарий по чек-листу требований.

Сценарий приёмки (демонстрация):

Шаг	Действие	Ожидаемый результат
1	`POST /api/v1/auth/login`	200, выдан токен, профиль с ролью
2	`GET /api/v1/equipment/folders`	список доступных папок
3	`POST /api/v1/equipment`	201, создана карточка прибора
4	`POST /api/v1/equipment/{id}/verification`	201, создана поверка
5	`POST /api/v1/equipment/{id}/verification/close`	поверка закрыта, статус обновлён
6	`POST /api/v1/arshin/search`	результаты поиска из ФГИС
7	`POST /api/v1/equipment/folders/{id}/refresh-tasks`	201, запущено пакетное обновление
8	`GET /api/v1/events`	события по выполненным действиям

Шаг	Действие	Ожидаемый результат
9	Вход под `CUSTOMER` и запрос чужой папки	404 (soft-denial)
10	`GET /docs`	OpenAPI-спецификация со всеми эндпоинтами

Критерии приёмки: все требования приложения А выполнены или явно отмечены как «не реализовано»; автоматические тесты зелёные; нагрузочный прогон без ошибок 5xx.

11.7 Требования к документированию

Комплект документации:

Документ	Состояние
Техническое задание (настоящий документ)	готов
Отчёт по практике 1: анализ предметной области	готов
ER-диаграмма и описание модели данных	готов
Схема БД (DDL PostgreSQL)	готов
Презентация клиент-серверной архитектуры	готов
README: описание, стек, запуск, тесты	готов (в репозитории)
OpenAPI-спецификация	генерируется автоматически (`/docs` , `/api/v1/openapi.json`)
Коллекция запросов `slumber.yml`	готов (интеграция с Аршином)
CHANGELOG	ведётся (v0.13.0)

Документы оформляются в Markdown, по требованию дисциплины экспортируются в PDF. Схема БД дополнительно поставляется исполняемым SQL-скриптом.

Приложение А. Функциональные требования

Приоритеты: **M** — Must Have, **S** — Should Have, **N** — Nice to Have. Столбец «Реализация» отражает фактическое состояние кода на 18.10.2026.

№	Требование	Приоритет	Реализация
F1	Аутентификация по email/паролю, выдача stateless-токена (HMAC)	M	реализовано
F2	Управление пользователями: создание, редактирование, блокировка, роль, временный пароль	M	реализовано
F3	Иерархия «папка → группа → оборудование» и CRUD каждого уровня	M	реализовано
F4	Карточка оборудования: тип (СИ/ЭСИ/ИО/ВО/прочее), наименование, модификация, заводской номер, год выпуска, диапазон, единица	M	реализовано

№	Требование	Приоритет	Реализация
	измерения, статус		
F5	Поиск, фильтрация, сортировка, пагинация по реестру	M	реализовано
F6	Разграничение доступа по ролям и папкам, soft-denial (404)	M	реализовано
F7	Поверка СИ: создание, режим, этапы, сроки, ответственные, закрытие	M	реализовано
F8	Ремонт: создание, этапы, сроки, ответственные, закрытие с результатом	M	реализовано
F9	Групповые (batch) операции по поверкам и ремонтам	S	реализовано
F10	Интеграция с ФГИС «Аршин»: поиск и детализация по свидетельству и рег. номеру	M	реализовано
F11	Пакетное обновление папки из Аршина с журналом по каждой строке	S	реализовано
F12	Комментарии, вложения (изображения/PDF), @упоминания	S	реализовано
F13	Email-уведомления (упоминания, временный пароль, события процессов), проверка SMTP	S	реализовано
F14	Журнал событий (audit log) по оборудованию, поверкам, ремонтам, задачам	S	реализовано
F15	Дашборд с виджетами по папкам, экспорт в XLSX	S	реализовано
F16	Подписка на события процесса по прибору или папке	N	реализовано
F17	Health/ready-эндпоинты для мониторинга	S	реализовано

Сверх перечисленного в практике 1 (появилось в ходе разработки):

№	Требование	Реализация
F18	Задачи: kanban-доска, участники с ролями, чек-листы, сроки, напоминания	реализовано
F19	Приватные комментарии и сообщения	реализовано
F20	Состав эталона (внутренние/внешние модули) и мониторинг ЭСИ	реализовано
F21	Импорт перечня СИ из Excel	реализовано
F22	Архив вложений поверки/ремонта в ZIP	реализовано

Приложение Б. Нефункциональные требования

Б.1 Производительность

Показатель	Значение
Целевая аудитория	десятки одновременных пользователей
Время отклика типовых GET	≤ 500 мс при нормальной нагрузке
Пагинация списков	обязательна для реестра, очередей, событий, задач

Показатель	Значение
Тяжёлые операции	выносятся в фон (RQ) или `BackgroundTasks`

Фактические замеры (Locust, production-конфигурация, 2026-10-06):

Метрика	30 users	60 users
Запросов / RPS	2 311 / 19,3	3 255 / 27,3
Ошибки	0	0
p50	53 мс	660 мс
p95	1,4 с	2,4 с

Узкое место — `POST /auth/login` (PBKDF2 600k итераций выполняется в event loop). Требование: вынести хеширование в пул потоков (`anyio.to_thread.run_sync`).

Б.2 Безопасность — см. 11.4.4.

Б.3 Надёжность — см. 11.4.3.

Б.4 Масштабируемость

- горизонтальное масштабирование backend (несколько uvicorn-воркеров/контейнеров);
- очередь Redis позволяет добавить воркеров без изменения API;
- stateless-аутентификация не требует sticky sessions.

Б.5 Совместимость

- браузеры: Chrome/Chromium, Firefox, Safari (две последние версии);
- СУБД: PostgreSQL 16 (допустимо 14+);
- ОС сервера: Linux; развёртывание в Docker.

Б.6 Ограничения

- фиксированный стек: Python/FastAPI/PostgreSQL/React;
- размещение преимущественно на собственном сервере;
- правовые: обработка персональных данных пользователей (152-ФЗ), публичные данные ФГИС «Аршин»;
- мобильное приложение не предусмотрено (адаптивный веб-клиент).

Б.7 Требования к коду и сопровождению

- линтеры: `ruff` (backend), `eslint` (frontend), `mypy` (типизация);
- тесты запускаются в CI до сборки образов;
- миграции только через Alembic, ручные правки схемы на проде запрещены;
- версия приложения — единый источник в `frontend/package.json` и корневом `package.json`.

Приложение В. Состав API (сводка)

Группа	Префикс	Назначение
Аутентификация	`/api/v1/auth`	вход, профиль, смена пароля, тест SMTP
Пользователи	`/api/v1/users`	CRUD, роли, сброс пароля, упоминания
Оборудование	`/api/v1/equipment`	папки, группы, карточки, поверки, ремонты, комментарии, вложения, экспорт
Аршин	`/api/v1/arshin`	статус, поиск, детализация СИ и ЭСИ
Задачи	`/api/v1/tasks`	CRUD, доска, чек-листы, сообщения, вложения, подписки
События	`/api/v1/events`	журнал, экспорт
Служебные	`/api/v1/health`	живость и готовность

Всего: **91 путь, 124 операции** (по данным `app.openapi()`). Спецификация доступна в `/docs` (Swagger UI), `/redoc` и `/api/v1/openapi.json`.

Согласовано:

Роль	Подпись	Дата
Студент, Макеев Г. Б.		18.10.2026
Преподаватель, Бучинский И. Н.		